VisEditBench: Can Vision-Language Models Edit Visualization Code from Multimodal Feedback?

Mizanur Rahman[‡ *], **Arshia Azimlu**[‡ †],
Shadikur Rahman[‡†], **Md Tahmid Rahman Laskar**[‡†], **Amran Bhuiyan**[‡†],
Shafiq Joty[$,¶], **Enamul Hoque Prince**[‡*]
[‡]York University
[$]Nanyang Technological University [¶]Salesforce AI Research

Abstract

Vision-language models (VLMs) have shown strong capabilities in generating visualization code from textual or visual specifications. However, real-world visualization authoring is inherently iterative: users frequently revise existing visualizations to repair flawed charts or adapt them to desired styles. Existing benchmarks primarily evaluate generation from scratch, leaving visualization code editing from multimodal feedback largely unexplored. We introduce **VisEditBench**, a benchmark of 1,395 human-annotated visualization code-editing tasks grounded in realistic visualization workflows and failure cases. VisEditBench covers two practical settings: *feedback-guided repair*, where models revise visualization code using buggy or marked charts together with textual feedback, and *reference-guided restyling*, where models modify code to match a target chart image. Evaluating 24 state-of-the-art VLMs reveals that visualization code editing remains challenging: Claude Opus 4.6 achieves the best overall pass rate of 76.0%, while most open-source models remain below 50%. Performance is particularly weak on visually grounded style adaptation, where Claude-4.6-Sonnet achieves only 55.71%. To establish a strong baseline, we further propose **VisEditAgent**, a render-grounded editing framework that iteratively generates, executes, validates, and refines candidate edits. Built on GPT-4o, VisEditAgent improves overall pass rate from 55.75% to 67.99%, demonstrating the importance of render-grounded feedback for faithful visualization editing. We will release VisEditBench at https://github.com/vis-nlp/VisEditBench.

1 Introduction

Data visualizations play a central role in modern data analysis and communication, enabling data scientists, business analysts, journalists, policymakers, and researchers to explore patterns, communicate insights, and support decision-making across domains (Hoque and Islam, 2024; Rahman et al., 2025a). To support increasingly complex visualization workflows, recent advances in vision-language models (VLMs) have driven rapid progress in automated visualization, including chart generation from natural-language instructions (Rahman et al., 2025b; Maddigan and Susnjak, 2023) and visualization code reconstruction from chart images (Yang et al., 2025; Wu et al., 2025).

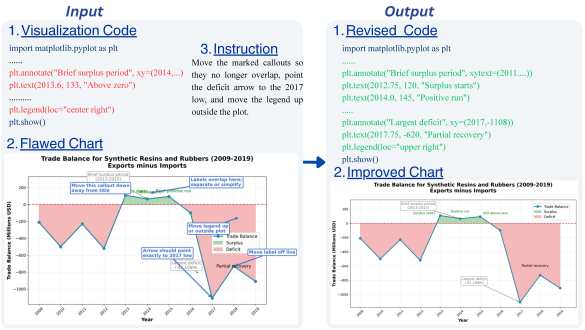


Figure 1: Overview of a VisEditBench task. Given visualization code, visual feedback or a reference chart, and a language instruction, the model must generate revised executable code that repairs or restyles the visualization while preserving the underlying data semantics.

Yet real-world visualization authoring rarely ends with the first generated chart. Even when models produce executable visualizations, the outputs often fail to fully match user intent, visual preferences, accessibility requirements, or publication constraints (Rahman et al., 2025b; Chen et al., 2024). As a result, users frequently inspect rendered charts, revise the underlying code, and rerender the output through iterative refinement. In many workflows, charts must also be adapted to match preferred visual styles or reference designs, since default visualizations are often stylistically homogeneous and poorly aligned with communicative goals. Figure 1 illustrates a typical repair workflow: although the user already has vi-

*Corresponding authors: {mizanurr,enamulh}@yorku.ca
†These authors contributed equally.

Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing, pages 23828–23846
October 24-29, 2026 ©2026 Association for Computational Linguistics

sualization code and a rendered chart, the output remains visually flawed; labels overlap, annotations obscure important regions, and visual emphasis is misplaced. Even seemingly minor edits can trigger cascading layout and styling changes, requiring repeated rounds of debugging and visual inspection. This repair-and-restyling cycle is one of the most common and time-consuming bottlenecks in practical visualization workflows (Harper and Agrawala, 2017). Unlike one-shot generation, these workflows require localized edits that preserve data semantics, layout structure, encodings, and analytical meaning while modifying only the desired visual properties. Visualization authoring is therefore fundamentally an *iterative multimodal editing problem*. Solving this problem requires grounded multimodal reasoning. A model must determine *what* is visually wrong, *which* code regions should change, *how* the rendered chart should evolve, and *which* aspects of the original visualization must remain preserved. This demands substantially richer reasoning than standard code generation or chart reconstruction.

Despite rapid progress in VLMs, existing benchmarks largely overlook this setting. Text-to-visualization benchmarks mainly evaluate chart generation from natural-language queries or data tables (Luo et al., 2021; Liu et al., 2021; Chen et al., 2024), while chart-to-code benchmarks evaluate code reconstruction from chart images (Yang et al., 2025; Wu et al., 2025). Neither captures a common workflow where users already have visualization code and want models to iteratively *edit* rendered visualizations using multimodal feedback or reference charts. More general multimodal software-engineering benchmarks (Yang et al., 2024b) study visual bug fixing, but do not address visualization-specific challenges such as repairing misleading encodings, resolving cluttered layouts, or matching reference chart styles while preserving data semantics.

To address this gap, we introduce **VisEditBench**, a benchmark for evaluating visualization code editing from multimodal feedback. Each task provides existing visualization code, a rendered chart, and a natural-language editing instruction, and requires the model to generate revised executable code that produces the desired visualization. VisEditBench supports two practical editing settings. In *feedback-guided repair*, models receive buggy or human-marked charts together with textual feedback and must repair the visualization while preserving its intended meaning (Fig. 1). In *reference-guided restyling*, models receive a target chart image and must adapt the original visualization to match the reference style without altering the underlying data semantics. Across these settings, VisEditBench contains 1,395 human-annotated tasks grounded in realistic visualization issues collected from Stack Overflow, Matplotlib and Vega-Lite issue reports, and diverse real-world visualization datasets.

Beyond executability, successful visualization editing requires edits that are visually faithful, semantically precise, and effective. To support fine-grained analysis, VisEditBench introduces a taxonomy of eight visualization-editing intents spanning correctness repair, readability improvement, style adaptation, consistency harmonization, robustness improvement, structural transformation, constraint satisfaction, and style-aware repair (Fig. 3). Together, these settings capture realistic visualization workflows requiring visual reasoning, code understanding, and multimodal grounding.

We evaluate 24 state-of-the-art VLMs on VisEditBench in a zero-shot setting using executability, task accuracy, readability and clarity, visual quality, and final pass rate. Results show that visualization editing remains challenging: Claude Opus 4.6 achieves the best overall pass rate at 76.0%, while most open-source models remain below 50%. Performance is particularly weak on visually grounded style adaptation, where even Claude-4.6-Sonnet achieves only 55.71%. To further study how rendering feedback can improve editing, we introduce **VisEditAgent**, a render-grounded baseline, that generates multiple candidate revisions, executes and renders them, visually validates the outputs, and iteratively refines the selected solution. Using GPT-4o as the base model, VisEditAgent improves overall pass rate from 55.75% to 67.99%, demonstrating that render-grounded feedback is critical for visually faithful visualization editing.

In summary, our contributions include: (*i*) **VisEditBench**, a benchmark of 1,395 human-annotated visualization code-editing tasks covering feedback-guided repair and reference-guided restyling across diverse visualization-editing intents; (*ii*) a structured **evaluation framework** measuring executability, task accuracy, chart readability and clarity, visual quality, visual similarity, and final pass rate; (*iii*) **zero-shot evaluations** of 24 state-of-the-art VLMs, revealing major limitations in executable and visually faithful chart editing; and (*iv*) **VisEditAgent**, a render-grounded

baseline that improves visualization editing through execution feedback, visual validation, and iterative refinement.

2 Related Work

Text-to-Visualization Existing visualization benchmarks and systems have primarily studied how models generate (Rahman et al., 2025b), understand (Hoque et al., 2022), or reconstruct charts (Wu et al., 2025) from natural-language queries, tabular data, chart images, and analytical intents (Tab. 6). WikiSQL (Zhong et al., 2017) supports natural-language-to-SQL parsing over tables, while nvBench (Luo et al., 2021) extends this setting to large-scale cross-domain NL2VIS by synthesizing paired natural-language queries and visualization specifications from NL-to-SQL benchmarks. Later systems explore visualization recommendation, specification generation, and code generation from free-form analytical queries (Liu et al., 2021; Dibia and Demiralp, 2019; Narechania et al., 2020; Song et al., 2022; Maddigan and Susnjak, 2023). Recent LLM-based benchmarks, including Text2Vis (Rahman et al., 2025b) and VisEval (Chen et al., 2024), evaluate whether large language models can generate visualizations across diverse real-world datasets. ChartBench (Xu et al., 2023) focuses on chart comprehension and visual reasoning, while ChartLlama (Han et al., 2023), ChartMimic (Yang et al., 2025), and Plot2Code (Wu et al., 2025) test chart reasoning or visualization-code reconstruction from chart images.

However, existing benchmarks do not evaluate iterative visualization editing from multimodal feedback, where users revise existing visualization code using rendered feedback or reference charts while preserving data semantics (Tab. 6). VisEditBench addresses this gap by evaluating visualization code editing from multimodal feedback through feedback-guided repair and reference-guided restyling tasks.

Multimodal Code Generation. Recent work has extended code generation to multimodal settings, where models generate or edit code from visual inputs (Li et al., 2024; Si et al., 2025; Wu et al., 2025). Early UI-to-code systems translated screenshots or sketches into executable interfaces (Beltramelli, 2018; Robinson, 2019), while later work improved screenshot-to-code generation using vision-code architectures and visual feedback (Soselia et al., 2023). More recent benchmarks study visually grounded programming tasks, including diagram-based programming (MMCode (Li et al., 2024)), screenshot-to-webpage generation (Design2Code (Si et al., 2025)), SVG editing (SVGEditBench (Nishina and Matsui, 2024)), and visual software bug fixing (SWE-bench Multimodal (Yang et al., 2024b)). In contrast, VisEditBench focuses on editing and restyling existing visualization code from multimodal feedback while preserving data semantics.

3 VisEditBench

We introduce VisEditBench, a benchmark of 1,395 human-annotated visualization code-editing tasks from multimodal feedback.

3.1 Data Collection

We designed VisEditBench to capture realistic visualization editing workflows rather than synthetic chart-generation prompts. To ensure both realism and diversity, we combine two complementary sources: real user-reported visualization issues and model-generated chart failures on real-world datasets. Fig. 2 illustrates the collection process.

First, we manually collected 120 real visualization-editing cases from Stack Overflow and Matplotlib/Vega-Lite GitHub issue discussions. We retained only examples containing a user-reported issue, visualization code, and a corresponding rendered buggy chart with clear visual problems requiring repair, improvement, or restyling. These naturally occurring cases include cluttered layouts, unreadable labels, malformed axes, misleading encodings, annotation failures, broken legends, and rendering inconsistencies.

Second, we use the Text2Vis benchmark (Rahman et al., 2025b), whose tables and queries are drawn from realistic data sources such as Statista (Statista, 2024), Pew Research (Pew, 2024), Our World in Data (OWID, 2024), and OECD (OECD, 2024). We reran both frontier and open-source models on Text2Vis queries, executed the generated code, and manually inspected the rendered charts to identify visual, semantic, and presentation failures. We retained chart-code pairs only when the rendered output exhibited clear visualization errors or imperfect visual design, such as layout problems, incorrect encodings, unreadable labels, or poor presentation quality. These retained cases therefore reflect realistic examples where current models fail in zero-shot visualization genera-

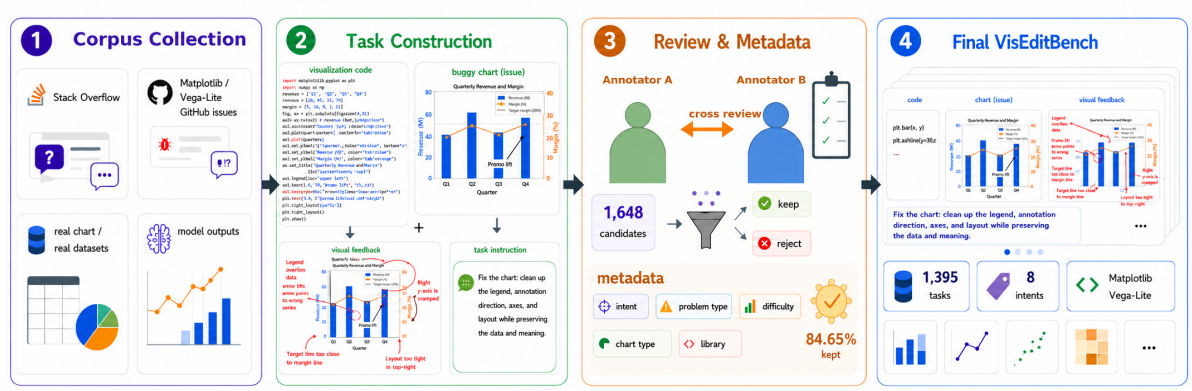


Figure 2: Overview of the VisEditBench construction pipeline. We collect real visualization issues and real-world chart/data examples, construct multimodal editing tasks, cross-review candidates for quality, and retain the final benchmark with metadata on editing intents, problem types, chart types, and visualization libraries.



Figure 3: Examples from VisEditBench across eight editing intents. Each task pairs input visualization code with a buggy, human-marked, or reference chart image and a natural-language instruction; the model must output revised executable code that renders the desired visualization.

tion. This process substantially increases diversity beyond naturally reported bugs, covering not only correctness repair, but also style adaptation, consistency harmonization, robustness improvement, structural transformation, and style-aware repair.

3.2 Annotation and Quality Control

From these sources, four annotators created an initial pool of 1,648 candidate editing tasks. For each task, annotators wrote a natural-language editing instruction and assigned metadata including editing intent, problem type, visualization library, and chart type. Each candidate was then independently cross-reviewed by another annotator to verify that the task was visually grounded, realistic, clearly specified, and solvable through code edits. A candidate was retained only when the reviewer judged that the code, chart image, instruction, and metadata together formed a valid editing task with an observable visual target and a feasible code-level solution.

To measure annotation reliability, we computed inter-annotator agreement on the binary accept/reject decision across all candidates. Annotators agreed on 84.65% of cases, indicating strong consensus on what constitutes a valid visualization editing task. We removed the remaining examples because they were ambiguous, insufficiently grounded in the image, duplicated, unrealistic, underspecified, or not clearly solvable through visualization code editing. This filtering process resulted in the final VisEditBench dataset of 1,395

Editing Intent (Count / %)								Problem Type (Count / %)				
Correct. Repair	Quality Improv.	Robust. Gen.	Style Adapt.	Refactor/ Transform.	Consistency Harmon.	Constraint Satis.	Style-aware Repair	Layout & Geometry	Annotation & Labeling	Encoding & Mapping	Data Transform.	Other
529 / 37.9	446 / 32.0	116 / 8.3	70 / 5.0	63 / 4.5	59 / 4.2	52 / 3.7	49 / 3.5	802 / 57.5	348 / 24.9	156 / 11.2	63 / 4.5	26 / 1.9

Cause Type (Count / %)		Complexity (Count / %)			Chart Type (Count / %)					Library (Count / %)	
Multi-cause	Single-cause	Easy	Medium	Hard	Bar	Line	Boxplot	Scatter	Other	Matplotlib	Vega-Lite
797 / 57.1	598 / 42.9	516 / 37.0	498 / 35.7	381 / 27.3	679 / 48.7	522 / 37.4	54 / 3.9	52 / 3.7	88 / 6.3	1156 / 82.9	239 / 17.1

Table 1: Statistics and diversity of VisEditBench. Counts and percentages are computed over the final 1,395 examples.

high-quality tasks. Each retained instance contains existing visualization code, a chart image, textual feedback or editing instruction, visual feedback or a reference chart, and structured metadata for fine-grained analysis.

3.3 Dataset Diversity

Table 1 summarizes the diversity of VisEditBench. The dataset spans eight editing intents, with Correctness Repair (37.9%) and Quality Improvement (32.0%) forming the largest categories, reflecting the prevalence of misleading, cluttered, or visually ineffective charts in real workflows. VisEditBench also covers diverse problem types and difficulty levels. Layout & Geometry issues are most common, followed by Annotation & Labeling, Encoding & Mapping, and Data Transformation. These categories include failures such as overlapping labels, malformed axes, incorrect encodings, broken annotations, and data-processing errors. Importantly, 57.1% of tasks are multi-cause, requiring coordinated edits across multiple chart components rather than isolated fixes. The dataset further spans varying levels of complexity, including a substantial number of challenging visually grounded editing problems (27.3% hard problems). Difficulty labels were assigned using GPT-5 under a fixed rubric and then manually reviewed and verified by human annotators (classification prompt is in Figure 7).

VisEditBench spans diverse chart types, editing behaviors, and visualization paradigms. Bar and line charts are the most common chart families, reflecting their prevalence in practical visualization workflows, while the benchmark also includes scatter plots, boxplots, heatmaps, treemaps, histograms, waterfall charts, tables, and composite visualizations. The benchmark further covers both imperative and declarative visualization paradigms through 1,156 Matplotlib examples and 239 Vega-Lite examples. Matplotlib tasks often require procedural edits involving layout control, annotation placement, and axis formatting, whereas Vega-Lite tasks require modifying declarative specifications such as encodings, scales, legends, layers, and configuration fields.

4 Methodology

4.1 Task Formulation

We formulate visualization code editing as a multimodal code revision task. Each example is defined as $x_i = (c_i, I_i, u_i, c_i')$, where c_i is the input visualization code, I_i is a rendered chart image, u_i is a natural-language editing instruction, and c_i' is the revised visualization code. Given (c_i, I_i, u_i), the model must generate executable code c_i' that produces the desired visualization while preserving the relevant data semantics.

VisEditBench supports two practical editing settings. In *feedback-guided repair*, I_i contains a flawed or human-marked chart, and the model must repair the visualization according to the provided feedback. In *reference-guided restyling*, I_i serves as a target chart or style reference, and the model must revise the original visualization to visually align with the reference while preserving its underlying analytical meaning.

4.2 Model Inference

We evaluate 24 state-of-the-art VLMs spanning both closed-source and open-source model families. Closed-source models include multimodal systems from the GPT, Gemini, and Claude families (OpenAI, 2024; Team, 2024), while open-source models cover recent VLM and code-oriented families including Qwen, InternVL, Pixtral, and Gemma (Yang et al., 2024a; Jiang et al., 2023; Grattafiori et al., 2024; DeepSeek-AI et al., 2024; Guo et al., 2025; Roziere et al., 2023). This setup enables direct comparison between frontier proprietary systems and deployable open-source alternatives under a unified visualization editing protocol. We use a fixed decoding configuration for all models to support reproducibility. Full inference and evaluation parameters are provided in App. A.2 (Tab. 10).

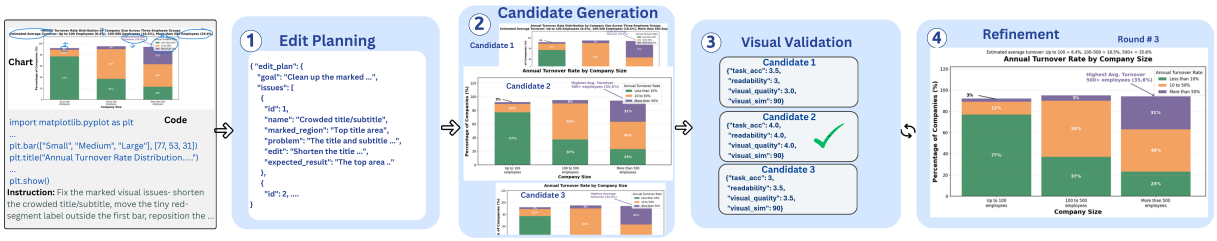


Figure 4: **VisEditAgent framework**. Given visualization code, a buggy, human-marked, or reference chart image, an instruction, and an edit intent, VisEditAgent plans the edit, generates candidate revisions, executes and renders them, validates the outputs, selects the best candidate, and refines it through iterative refinement rounds.

4.3 VisEditAgent

While zero-shot prompting can produce plausible visualization edits, real-world visualization authoring is inherently iterative (Munzner, 2014): authors repeatedly modify code, render charts, inspect the output, and refine the visualization through successive feedback loops. This process is especially important for visually grounded edits involving layout, annotations, encodings, and style, where small code changes can trigger cascading visual errors or distort underlying data semantics. Motivated by this workflow, we introduce **VisEditAgent**, a strong render-grounded baseline for visualization code editing. Rather than relying on a single-pass response, VisEditAgent iteratively generates, renders, validates, and refines candidate edits using execution and visual feedback (Fig. 4).

(1) Edit Planning. Given the input code, chart image, instruction, and editing intent, the framework first identifies the likely visual issue, relevant code regions, and required modification strategy. This stage grounds the edit in both the rendered visualization and the underlying code structure.

(2) Candidate Generation. VisEditAgent generates multiple candidate code edits representing different implementations of the requested modification. Each candidate is then executed and rendered into a chart image, enabling the framework to jointly evaluate program-level correctness and visualization-level quality. Exploring multiple rendered candidates is particularly important because many visualization edits admit several plausible yet visually different solutions.

(3) Visual Validation and Selection. Rendered candidates are evaluated using the task instruction, input chart, and reference or target image. The framework assesses whether each candidate satisfies the requested edit, preserves the intended data semantics, and visually aligns with the desired output. VisEditAgent then selects the strongest candidate based on execution success and visual alignment.

(4) Feedback-Based Refinement. The selected candidate is further refined using execution diagnostics, rendering failures, and visual-validation feedback. This refinement stage helps correct common one-shot failure modes.

5 Evaluation

5.1 Evaluation Criteria

Evaluating visualization editing requires measuring not only code executability, but also whether the edited chart satisfies the intended visual revision while remaining readable, visually coherent, and faithful to the target output. We therefore evaluate each output using five criteria: (i) **Code Execution**, which checks whether the generated code executes successfully and renders a valid chart; non-executable, timed-out, or empty outputs receive zero scores for all remaining metrics; (ii) **Task Accuracy**, which measures whether the edited chart satisfies the requested instruction or matches the target/reference chart while preserving the intended visualization semantics; (iii) **Readability and Clarity**, which evaluates the legibility and organization of labels, legends, axes, ticks, annotations, and other chart elements; (iv) **Visual Quality**, which assesses overall presentation quality, including layout, scaling, color usage, chart appropriateness, and visual polish; and (v) **Visual Similarity**, which measures how closely the rendered output aligns with the corrected or reference chart image.

Automatic Evaluator. We use a rubric-based VLM evaluator with fixed prompts, deterministic settings, explicit scoring criteria, and a fixed input order. The evaluator receives the task instruction, the input or reference chart, and the rendered model output, without access to model identities. By default, GPT-4o serves as the evaluator for all visual metrics; when evaluating GPT-4o outputs, we instead use Gemini 2.5 Pro to avoid self-evaluation bias.

Model	Correctness Repair	Quality Improvement	Robustness / Generalization	Style Adaptation	Constraint Satisfaction	Consistency Harmonization	Refactor / Transformation	Style-Aware Error Repair	Overall
Closed-Source Models									
GPT-4o	56.08	67.34	57.76	10.00	63.46	47.46	42.86	30.61	55.75
GPT-5-Mini	55.51	64.64	71.55	47.14	67.30	55.93	58.73	51.02	60.00
GPT-5	61.60	75.45	56.89	**57.14**	59.61	**66.10**	69.84	51.02	65.68
Gemini-3.0-Flash	17.96	19.28	9.48	35.71	5.77	15.25	9.52	0.00	17.06
Gemini-2.5-Pro	61.20	70.90	73.30	38.60	69.20	52.50	57.10	40.80	63.40
Claude-Haiku-4.5	58.40	66.80	67.20	31.40	65.30	49.20	41.20	36.70	58.80
Claude-4.5-Sonnet	60.83	65.34	75.00	52.86	67.31	42.38	74.60	51.02	62.73
Claude-4.6-Sonnet	**70.34**	**79.73**	**86.20**	55.71	**80.76**	64.40	**85.71**	61.22	74.46
Claude-Opus-4.1	64.50	73.50	80.10	48.60	73.10	49.20	74.60	46.90	67.30
Claude-Opus-4.6	69.90	79.10	84.50	**80.00**	79.10	59.30	79.40	**77.60**	**76.00**
Open-Source Models									
Gemma-3-4B	30.79	25.00	1.00	0.00	3.85	16.95	3.18	1.00	21.29
Gemma-3-12B	42.97	47.97	6.89	1.43	13.46	28.81	7.94	10.20	34.96
Gemma-3-27B	49.24	56.76	23.28	0.00	26.92	42.37	12.70	12.25	42.95
InternVL-3.5-1B	11.41	7.88	0.00	0.00	1.92	8.47	1.59	2.04	7.41
InternVL-3.5-2B	25.67	21.85	0.00	0.00	3.85	20.34	0.00	6.12	18.13
InternVL-3.5-4B	34.03	33.78	8.62	0.00	9.61	28.81	1.58	2.04	26.33
InternVL-3.5-8B	42.96	41.22	7.75	0.00	11.54	32.20	12.70	10.20	33.17
InternVL-3.5-14B	41.65	49.78	3.45	0.00	3.85	37.29	7.94	10.21	34.83
Pixtral-12B	44.68	47.07	5.17	0.00	15.38	30.51	6.35	10.20	35.32
Qwen3-VL-2B	24.52	20.49	1.00	0.00	5.77	16.95	7.94	6.12	17.55
Qwen3-VL-4B	51.11	50.90	1.72	1.42	7.69	37.28	4.76	16.32	38.84
Qwen3-VL-8B	49.04	52.02	28.44	0.00	5.70	52.44	26.98	14.29	42.09
Qwen3-VL-30B-A3B	49.80	53.38	44.83	15.71	38.46	42.37	47.61	20.41	47.05
Qwen3-VL-32B	54.95	59.00	50.00	21.42	48.10	54.23	36.51	16.32	51.72

Table 2: Pass rate by editing intent on VisEditBench. All values are percentages (%). Overall averages all examples.

We further validate the evaluator through human assessment using the same rubric and report agreement between automatic and human judgments.

Scoring and Final Pass Rate. Task accuracy, readability and clarity, and visual quality are scored on a 0–5 scale with 0.5-point increments, while visual similarity is scored from 0–100. We additionally report a strict final pass rate: an example passes only if the code executes successfully, task accuracy is at least 4.5, readability and clarity and visual quality are at least 4.0, and visual similarity is at least 90. These thresholds intentionally measure complete task success rather than partial improvement, ensuring that edited charts are executable, visually coherent, instruction-faithful, and aligned with the intended target. The full evaluation rubric, including score definitions and intent-specific visual similarity rules, is provided in Table 9.

5.2 Main Results

5.2.1 Zero-Shot Performance

Tables 2 and 3 reveal a substantial gap between generating executable visualization code and performing visually faithful visualization editing. Overall, closed-source models substantially outperform open-source models, but even the strongest model remains far from solving visualization code editing. Claude Opus 4.6 achieves the best overall pass rate at 76.0%, followed by Claude-4.6-Sonnet at 74.46% and Claude Opus 4.1 at 67.3%. GPT-5 follows closely at 65.68%. Among open-source models, Qwen3-VL-32B performs best with 51.72%, while most open-source models remain below 50%. Smaller models perform much worse, such as InternVL-3.5-1B at 7.41% and Qwen3-VL-2B at 17.55%, suggesting that visualization editing requires strong multimodal and reasoning ability. We verified that Gemini-3.0-Flash's unusually low performance is not due to a harness or parsing issue; under the shared generation budget, the model frequently produced lengthy reasoning-style outputs, resulting in some truncated or incomplete responses.

Model	Code Exec.	Task Acc.	Read. & Clarity	Visual Quality	Visual Similarity
Closed-Source Models					
GPT-4o	93.31	4.04	4.03	3.95	78.13
GPT-5-Mini	88.20	4.11	4.12	4.07	80.65
GPT-5	94.10	4.13	4.10	4.09	84.13
Gemini-3.0-Flash	46.02	1.83	2.07	2.04	42.85
Gemini-2.5-Pro	93.19	4.24	4.23	4.20	83.58
Claude-Haiku-4.5	91.97	4.12	4.11	4.06	80.90
Claude-4.5-Sonnet	94.46	4.41	4.38	4.38	86.52
Claude-4.6-Sonnet	96.19	4.57	4.54	4.55	89.42
Claude-Opus-4.1	94.70	4.43	4.40	4.40	86.80
Claude-Opus-4.6	**96.42**	**4.60**	**4.57**	**4.59**	**90.90**
Open-Source Models					
Gemma-3-4B	82.30	2.70	3.30	3.33	64.78
Gemma-3-12B	87.19	3.57	3.75	3.78	74.20
Gemma-3-27B	88.27	3.92	3.97	3.98	77.34
InternVL-3.5-1B	72.81	1.36	2.62	2.65	53.74
InternVL-3.5-2B	80.00	2.39	3.09	3.13	61.47
InternVL-3.5-4B	78.71	3.01	3.28	3.31	65.35
InternVL-3.5-8B	83.38	3.47	3.64	3.65	71.25
InternVL-3.5-14B	78.95	3.34	3.46	3.48	68.12
Pixtral-12B	86.04	3.46	3.66	3.69	72.43
Qwen3-VL-2B	79.93	2.24	3.09	3.13	62.48
Qwen3-VL-4B	79.71	3.08	3.39	3.43	67.86
Qwen3-VL-8B	85.83	3.68	3.79	3.82	74.77
Qwen3-VL-30B-A3B	89.71	3.92	4.03	4.05	79.19
Qwen3-VL-32B	87.77	4.05	4.01	4.04	78.38

Table 3: Metric-level scores on VisEditBench. Executability is reported as a percentage, Visual Similarity on a 0–100 scale, and other metrics on a 5-point scale.

A key finding is that executability alone is not the

Model / Method	Correct. Repair	Quality Improve.	Robust. Gen.	Style Adapt.	Constraint Satis.	Consistency Harmon.	Refactor/ Transform.	Style-aware Repair	Overall
GPT-4o Zero-shot	56.08	67.34	57.76	10.00	63.46	47.46	42.86	30.61	55.75
GPT-4o + VisEditAgent	64.64	**78.37**	**61.22**	**62.85**	**65.38**	**64.41**	**60.32**	**44.89**	**67.99**
Qwen3-VL-4B Zero-shot	51.11	50.90	1.72	1.42	7.69	37.28	4.76	16.32	38.84
Qwen3-VL-4B + VisEditAgent	63.14	63.00	42.24	35.71	44.23	54.23	42.85	18.36	44.98
Qwen3-VL-8B Zero-shot	49.04	52.02	28.44	0.00	5.70	52.44	26.98	14.29	42.09
Qwen3-VL-8B + VisEditAgent	**71.08**	67.71	1.80	24.29	13.46	49.15	17.46	16.36	54.53
Ablations on GPT-4o + VisEditAgent									
w/o Multi-candidate Generation	61.44	74.44	56.03	28.57	59.62	55.93	52.38	38.78	61.94
w/o Refinement	58.79	71.52	52.59	41.43	57.69	54.24	49.21	34.70	59.86

Table 4: Pass rate by editing intent for zero-shot models, VisEditAgent, and ablations. All values are percentages (%).

primary bottleneck for strong models. Claude Opus 4.6 and Claude-4.6-Sonnet execute successfully on 96.42% and 96.19% of examples, respectively, yet achieve lower visual similarity scores of 90.90 and 89.42. This indicates that models can often generate runnable code while still failing to preserve visual semantics, layout fidelity, or stylistic alignment with the target edit.

Performance also varies sharply across editing intents. Stronger models perform relatively well on correctness repair and quality improvement, where edits often involve localized readability or layout fixes. For example, Claude-4.6-Sonnet reaches 70.34% on correctness repair and 79.73% on quality improvement, while GPT-4o reaches 56.08% and 67.34%, respectively. In contrast, visually grounded and coordinated edits remain difficult: Claude-4.6-Sonnet achieves only 55.71% on style adaptation, and GPT-4o only 10.00%. Overall, VisEditBench requires more than code generation or chart reconstruction: successful models must jointly reason over code, visual feedback, user intent, and rendered chart quality.

Performance further decreases consistently with task difficulty, while variation across chart types reveals model-specific strengths and weaknesses; full breakdowns are provided in Appendix A.5 (Tab. 12, Tab. 13).

5.2.2 VisEditAgent Results

Table 4 shows that VisEditAgent consistently improves over direct zero-shot prompting. Using GPT-4o as the base model, VisEditAgent increases overall pass rate from 55.75% to 67.99%, demonstrating the importance of render-grounded iterative refinement for visualization editing. The largest gains appear on visually grounded editing tasks. Style adaptation improves from 10.00% to 62.85%, consistency harmonization from 47.46% to 64.41%, and refactor/transformation from 42.86% to 60.32%. Similar trends hold for open-source models: for example, Qwen3-VL-4B improves from 1.42% to 35.71% on style adaptation and from 4.76% to 42.85% on refactor/transformation. Table 5 further shows consistent improvements in task accuracy, readability, visual quality, and visual similarity. These results suggest that a central failure mode of zero-shot VLMs is not merely code generation, but the inability to iteratively validate and refine edits against rendered visual feedback. Multi-candidate generation, rendering, visual validation, and refinement substantially improve the ability of models to produce edits that are both executable and visually faithful (see Figure 5).

To validate the automatic evaluation, we manually evaluated outputs from GPT-4o zero-shot and GPT-4o + VisEditAgent on all 1,395 VisEditBench samples, resulting in 2,790 total model outputs. We additionally evaluated Qwen3-VL-4B zero-shot and Qwen3-VL-4B + VisEditAgent on a 500-example stratified sample. Five annotators assessed each output using the same rubric as the automatic evaluator, covering task accuracy, readability and clarity, visual quality, visual similarity, and final pass/fail judgment. As shown in Table 7, human evaluation confirms that VisEditAgent improves pass rate for both GPT-4o and Qwen3-VL-4B. The human scores also show strong agreement with the automatic evaluator, with Pearson correlations ranging from 83.38 to 87.00 across the four scalar metrics: task accuracy, readability and clarity, visual quality, and visual similarity (Table 8). This suggests that the automatic evaluation is well aligned with human judgments while enabling scalable assessment of visualization code editing performance.

As an additional robustness check, replacing the original candidate validator with Gemini 2.5 Pro yields a 69.40% pass rate, compared with 67.99% under the original setup, suggesting that VisEditAgent's gains are not tied to a particular validator

Model / Method	Code Exec.	Task Acc.	Read. & Clarity	Visual Quality	Visual Similarity
GPT-4o Zero-shot	93.31	4.04	4.03	3.95	78.13
GPT-4o + VisEditAgent	**96.83**	**4.37**	**4.35**	**4.36**	**86.30**
Qwen3-VL-4B Zero-shot	79.71	3.08	3.39	3.43	67.86
Qwen3-VL-4B + VisEditAgent	91.93	3.70	3.92	3.94	77.30
Qwen3-VL-8B + VisEditAgent	93.45	4.03	4.13	4.16	76.50

Table 5: Metric-level comparison of zero-shot models and VisEditAgent. Executability is reported as a percentage (%). Task Accuracy, Chart Readability and Clarity, and Visual Quality are reported on a 5-point scale. Visual Similarity is reported on a 0–100 scale. Best results in each column are shown in bold.

model.

In practice, VisEditAgent uses approximately five model calls per task (compared with one for zero-shot prompting), with an estimated GPT-4o API cost of CAD $0.08–$0.10 and an average latency of approximately 30 seconds per task.

5.3 Ablation Studies

As shown in Tab. 4, removing multi-candidate generation lowers VisEditAgent's overall pass rate with GPT-4o from 67.99% to 61.94%, while removing refinement lowers it to 59.86%. The largest drop occurs for style adaptation, where performance decreases from 62.85% to 28.57% and 41.43%, respectively. These results show that both candidate selection and feedback-based refinement are important for visually faithful editing. We further compare VisEditAgent against compute-matched baselines on a stratified 500-task subset; the full VisEditAgent achieves 68.0%, outperforming best-of-3 (58.2%), best-of-5 (59.4%), and code-only refinement (62.8%) (Appendix A.3, Tab. 11). We additionally find that removing the visual input reduces GPT-4o's pass rate from 59% to 42% on a stratified 500-task subset, confirming the importance of visual grounding (Appendix A.4).

5.4 Error Analysis

We conducted a qualitative error analysis on 500 randomly selected model outputs. We identify four recurring failure modes, illustrated in Fig. 6.

Executable but visually incorrect outputs. The code executes and renders a chart, but the edit is not satisfied. Typical failures include unresolved label overlap, misplaced legends, cluttered annotations, and poor readability.

Weak grounding in visual feedback. Models fail to localize edits from marked or reference images. They move annotations, legends, or labels to incorrect regions, or keep arrows pointing to the wrong data points.

Poor reference-style matching. In style adaptation, models often make generic visual changes rather than preserving the reference layout, colors, fonts, legends, backgrounds, or annotation style.

Incorrect or incomplete transformations. For refactor/transformation tasks, models change the wrong chart type, omit required visual elements, alter encodings, or fail to preserve data semantics.

Across the analyzed failures, 46.7% are executable but visually incorrect, 28.4% reflect weak grounding in visual feedback, 14.7% involve poor reference-style matching, and 9.7% correspond to incorrect or incomplete transformations. These results show that the dominant failure mode is not code execution itself, but producing edits that are visually faithful to the requested change.

6 Conclusion

We introduce **VisEditBench**, the first benchmark for visualization code editing from multimodal feedback, moving beyond the one-shot chart-generation focus of prior work. Across 1,395 human-annotated tasks spanning feedback-guided repair and reference-guided restyling, our evaluation reveals a substantial gap between generating executable charts and performing visually faithful visualization editing, particularly for visually grounded and coordinated edits requiring layout, style, and semantic preservation. As a strong baseline, we include **VisEditAgent**, a render-grounded framework that iteratively generates, executes, validates, and refines candidate edits using visual feedback. This framework substantially improves editing performance, demonstrating that reliable visualization editing requires iterative multimodal reasoning rather than single-pass code generation. Together, VisEditBench and VisEditAgent establish a strong foundation for advancing visually grounded, feedback-aware, and user-aligned visualization authoring systems.

Limitations

VisEditBench currently covers two visualization libraries, Matplotlib and Vega-Lite. These libraries represent both imperative and declarative visualization paradigms and support many common chart-editing scenarios. However, the benchmark does not yet directly evaluate other widely used visualization ecosystems such as Plotly, D3.js, and ggplot2. Extending VisEditBench to these libraries would allow future work to study whether the ob-

served model behaviors generalize across a broader range of visualization programming environments.

Although VisEditBench includes diverse chart types, editing intents, problem types, and difficulty levels, it is not intended to exhaustively cover every possible visualization-editing need. The benchmark emphasizes common editing scenarios such as layout adjustment, label and annotation repair, encoding correction, style adaptation, and reference-guided restyling. More specialized domains may involve additional constraints, domain conventions, or visualization types that are not fully represented in the current dataset.

Ethical Considerations

VisEditBench is designed to support transparent and reproducible research on multimodal visualization code editing. The benchmark is constructed from publicly available visualization resources, including user-reported visualization issues and real-world chart/data examples, and is used only for research and evaluation purposes. During dataset construction, annotators retained only examples with concrete visualization code, clearly grounded visual feedback, and implementable editing goals. No external or paid annotators were involved in this study; instead, all individuals who contributed to dataset annotation were granted co-authorship in recognition of their contributions. All annotators were informed that their annotations would be included in the dataset released for research purposes. We used LLM assistance only for language editing and polishing, while all technical content, dataset construction, experiments, and analyses were reviewed and verified by the authors.

To improve data quality and reduce ambiguity, each candidate task was cross-reviewed before inclusion in the final benchmark. We removed examples that were underspecified, duplicated, unrealistic, insufficiently grounded in the chart image, or not clearly solvable through visualization code editing. The benchmark does not aim to collect or evaluate sensitive personal information; its focus is on visualization code, rendered charts, editing instructions, and structured metadata.

We maintained fairness in model comparisons by applying the same prompting protocol, execution environment, evaluation criteria, and scoring rubrics across open-source and closed-source models. For automatic evaluation, we used a fixed rubric and avoided self-evaluation bias by using a different evaluator when assessing outputs from the default evaluator model. We also report limitations and failure modes to help prevent overclaiming model capabilities. Overall, VisEditBench is intended to encourage reliable, visually grounded, and user-aligned visualization authoring systems. Finally, we used AI-based writing assistants only to improve the presentation of the paper.

Acknowledgements

This research was supported by the Natural Sciences and Engineering Research Council (NSERC), Canada, Canada Foundation for Innovation, Compute Canada, and the CIRC grant on Inclusive and Accessible Data Visualizations and Analytics.

References

Tony Beltramelli. 2018. pix2code: Generating code from a graphical user interface screenshot. In *Proceedings of the ACM SIGCHI symposium on engineering interactive computing systems*, pages 1–6.

Nan Chen, Yuge Zhang, Jiahang Xu, Kan Ren, and Yuqing Yang. 2024. Viseval: A benchmark for data visualization in the era of large language models. *IEEE Transactions on Visualization and Computer Graphics*.

DeepSeek-AI, Qihao Zhu, Daya Guo, Zhihong Shao, Dejian Yang, Peiyi Wang, Runxin Xu, Y. Wu, Yukun Li, Huazuo Gao, Shirong Ma, Wangding Zeng, Xiao Bi, Zihui Gu, Hanwei Xu, Damai Dai, Kai Dong, Liyue Zhang, Yishi Piao, Zhibin Gou, Zhenda Xie, Zhewen Hao, Bingxuan Wang, Junxiao Song, Deli Chen, Xin Xie, Kang Guan, Yuxiang You, Aixin Liu, Qiushi Du, Wenjun Gao, Xuan Lu, Qinyu Chen, Yaohui Wang, Chengqi Deng, Jiashi Li, Chenggang Zhao, Chong Ruan, Fuli Luo, and Wenfeng Liang. 2024. Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence. *CoRR*, abs/2406.11931.

Victor Dibia and Çağatay Demiralp. 2019. Data2vis: Automatic generation of data visualizations using sequence-to-sequence recurrent neural networks. *IEEE computer graphics and applications*, 39(5):33–46.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.

Yucheng Han, Chi Zhang, Xin Chen, Xu Yang, Zhibin Wang, Gang Yu, Bin Fu, and Hanwang Zhang. 2023. Chartllama: A multimodal llm for chart understanding and generation. *arXiv preprint arXiv:2311.16483*.

Jonathan Harper and Maneesh Agrawala. 2017. Converting basic d3 charts into reusable style templates. *IEEE transactions on visualization and computer graphics*, 24(3):1274–1286.

Enamul Hoque and M Saidul Islam. 2024. Natural language generation for visualizations: State of the art, challenges and future directions. In *Computer Graphics Forum*, page e15266. Wiley Online Library.

Enamul Hoque, Parsa Kavehzadeh, and Ahmed Masry. 2022. Chart question answering: State of the art and future directions. In *Computer Graphics Forum*, volume 41, pages 555–572. Wiley Online Library.

Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. 2023. Mistral 7b. *arXiv preprint arXiv:2310.06825*.

Kaixin Li, Yuchen Tian, Qisheng Hu, Ziyang Luo, Zhiyong Huang, and Jing Ma. 2024. Mmcode: Benchmarking multimodal large language models for code generation with visually rich programming problems. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 736–783.

Can Liu, Yun Han, Ruike Jiang, and Xiaoru Yuan. 2021. Advisor: Automatic visualization answer for natural-language question on tabular data. In *2021 IEEE 14th Pacific Visualization Symposium (PacificVis)*, pages 11–20. IEEE.

Yuyu Luo, Jiawei Tang, and Guoliang Li. 2021. nvbench: A large-scale synthesized dataset for cross-domain natural language to visualization task. *arXiv preprint arXiv:2112.12926*.

Paula Maddigan and Teo Susnjak. 2023. Chat2vis: Generating data visualizations via natural language using chatgpt, codex and gpt-3 large language models. *Ieee Access*, 11:45181–45193.

Tamara Munzner. 2014. *Visualization Analysis and Design*, 1 edition. A K Peters/CRC Press.

Arpit Narechania, Arjun Srinivasan, and John Stasko. 2020. Nl4dv: A toolkit for generating analytic specifications for data visualization from natural language queries. *IEEE Transactions on Visualization and Computer Graphics*, 27(2):369–379.

Kunato Nishina and Yusuke Matsui. 2024. Svgeditbench: A benchmark dataset for quantitative assessment of llm's svg editing capabilities. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8142–8147.

OECD. 2024. Our world in data.

OpenAI. 2024. Gpt-4 technical report.

OWID. 2024. Our world in data.

Pew. 2024. Pew research center.

Mizanur Rahman, Amran Bhuiyan, Mohammed Saidul Islam, Md Tahmid Rahman Laskar, Ridwan Mahbub, Ahmed Masry, Shafiq Joty, and Enamul Hoque. 2025a. Llm-based data science agents: A survey of capabilities, challenges, and future directions. *arXiv preprint arXiv:2510.04023*.

Mizanur Rahman, Md Tahmid Rahman Laskar, Shafiq Joty, and Enamul Hoque. 2025b. Text2vis: A challenging and diverse benchmark for generating multimodal visualizations from text. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 31837–31862.

Alex Robinson. 2019. Sketch2code: Generating a website from a paper mockup. *arXiv preprint arXiv:1905.13750*.

Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*.

Chenglei Si, Yanzhe Zhang, Ryan Li, Zhengyuan Yang, Ruibo Liu, and Diyi Yang. 2025. Design2code: Benchmarking multimodal code generation for automated front-end engineering. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 3956–3974.

Yuanfeng Song, Xuefang Zhao, Raymond Chi-Wing Wong, and Di Jiang. 2022. Rgvisnet: A hybrid retrieval-generation neural framework towards automatic data visualization generation. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 1646–1655.

Davit Soselia, Khalid Saifullah, and Tianyi Zhou. 2023. Learning ui-to-code reverse generator using visual critic without rendering. *arXiv preprint arXiv:2305.14637*.

Statista. 2024. Statista.

Gemini Team. 2024. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context.

Chengyue Wu, Zhixuan Liang, Yixiao Ge, Qiushan Guo, Zeyu Lu, Jiahao Wang, Ying Shan, and Ping Luo. 2025. Plot2code: A comprehensive benchmark for evaluating multi-modal large language models in code generation from scientific plots. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 3006–3028.

Zhengzhuo Xu, Sinan Du, Yiyan Qi, Chengjin Xu, Chun Yuan, and Jian Guo. 2023. Chartbench: A benchmark for complex visual reasoning in charts. *arXiv preprint arXiv:2312.15915*.

An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. 2024a. Qwen2. 5 technical report. *arXiv preprint arXiv:2412.15115*.

Cheng Yang, Chufan Shi, Yaxin Liu, Bo Shui, Junjie Wang, Mohan Jing, Linran Xu, Xinyu Zhu, Siheng Li, Yuxiang Zhang, et al. 2025. Chartmimic: Evaluating lmm's cross-modal reasoning capability via chart-to-code generation. In *International Conference on Learning Representations*, volume 2025, pages 26590–26646.

John Yang, Carlos E Jimenez, Alex L Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R Narasimhan, et al. 2024b. Swe-bench multimodal: Do ai systems generalize to visual software domains? *arXiv preprint arXiv:2410.03859*.

Victor Zhong, Caiming Xiong, and Richard Socher. 2017. Seq2sql: Generating structured queries from natural language using reinforcement learning. *arXiv preprint arXiv:1709.00103*.

A Appendices

A.1 Annotation Guidelines and Quality Control

Before large-scale annotation, two domain experts defined the VisEditBench taxonomy, including the eight editing intents and major visualization problem types. They additionally curated 120 realistic seed examples from Stack Overflow and Matplotlib/Vega-Lite GitHub issue discussions, which served as reference examples for annotation. These examples guided four annotators in constructing complete candidate editing tasks from visualization failures. For each candidate, annotators identified the relevant visual, semantic, or presentation issue; prepared the corresponding code–chart pair and visual markings when applicable; wrote a natural-language editing instruction; and assigned the associated task metadata.

Annotation Guidelines. The central guideline was to construct a task only when the visualization issue was visually grounded, clearly describable, and solvable through code edits while preserving the underlying data semantics. Editing instructions were therefore required to correspond to an observable visualization issue and a concrete editing objective, rather than an underspecified request for general improvement. Depending on the task, annotators also included visual markings to localize the relevant region or element of the chart.

Cross-Review and Disagreement Handling. Each completed candidate task was independently cross-reviewed by another annotator. A candidate was retained only when both annotators accepted it as a valid visualization-editing task. Annotators agreed on 84.65% of candidate decisions. This quality-control process reduced the initial pool of 1,648 candidate tasks to the final set of 1,395 VisEditBench tasks. Candidates for which the required editing objective could not be established with sufficient clarity were excluded rather than resolved through post-hoc modification.

Rejected Cases. Candidates were rejected when they did not provide a sufficiently clear, visually grounded, and solvable editing objective. Common rejected cases included vague instructions such as "make the chart better," unclear visual annotations, duplicate tasks, mismatches between the visualization code and the corresponding rendered image, and cases that were not clearly solvable through

visualization-code editing. These filtering criteria were used to ensure that retained tasks admit a meaningful and reproducible evaluation of the requested edit.

Annotation Procedure. Annotators were guided by the benchmark taxonomy and the 120 curated seed examples throughout task construction. For each candidate, they inspected the visualization and associated code, identified the visual or semantic issue, prepared visual markings when necessary, formulated the corresponding editing instruction, and assigned task metadata. The resulting task was then passed to the independent cross-review stage described above. Thus, benchmark construction combined structured annotation guidance with independent human validation before inclusion in the final dataset.

A.2 Inference and Evaluation Parameters

To support reproducibility, we report the decoding, rendering, and evaluation parameters used in our experiments in Table 10.

A.3 Compute-Matched VisEditAgent Analysis

To separate the benefits of render-grounded editing from those of additional sampling and model calls, we further compare VisEditAgent against compute-matched baselines on a stratified subset of 500 VisEditBench tasks. Best-of-3 and best-of-5 sampling improve GPT-4o from 55.0% to 58.2% and 59.4%, respectively, while three-candidate code-only refinement reaches 62.8%. The full VisEditAgent achieves 68.0%, indicating that its gains are not explained solely by additional sampling or generic iterative refinement.

A.4 Visual-Input Ablation

To quantify the contribution of visual input, we conduct an image-removal ablation on a stratified subset of 500 VisEditBench tasks using GPT-4o zero-shot under the same prompting and evaluation protocol. We compare the full multimodal setting against a text-only setting in which the chart or reference image is removed. Removing the image decreases the pass rate from 59% to 42%, demonstrating that the visual input provides substantial information beyond the source code and textual instruction.

A.5 Performance by Difficulty and Chart Type

We further analyze model performance across task difficulty and chart type. Performance consistently decreases as task difficulty increases, indicating that harder editing tasks remain substantially more challenging even for frontier models. In contrast, performance across chart types is less uniform, with different models exhibiting distinct strengths across bar, line, scatter, and boxplot visualizations.

Benchmark	Main Task	Input	Code Given	Human Feedback	Code Editing	Ref. Restyling	Edit Taxonomy
Text2Vis (Rahman et al., 2025b)	Text-to-vis generation	Text + data	△	✗	✗	✗	✗
VisEval (Chen et al., 2024)	NL2VIS evaluation	Text + data	✗	✗	✗	✗	✗
ChartMimic (Yang et al., 2025)	Chart-to-code generation	Chart image	✗	✗	✗	✗	✗
Plot2Code (Wu et al., 2025)	Plot-to-code generation	Plot image	△	✗	✗	✗	✗
SWE-bench Multimodal (Yang et al., 2024b)	Visual software repair	Issue + repo.	✓	✗	✗	✗	✗
VisEditBench	**Visualization repair & restyling**	**Code + multimodal feedback**	✓	✓	✓	✓	✓

Table 6: Comparison of VisEditBench with related visualization and multimodal software benchmarks. ✓ = yes, ✗ = no, and △ = partial. VisEditBench uniquely evaluates editing existing visualization code from multimodal feedback, including feedback-guided repair and reference-guided restyling.

Model / Method	Code Execution	Task Accuracy	Readability & Clarity	Visual Quality	Visual Similarity	Pass Rate
GPT-4o Zero-shot	95.00	4.13	4.10	4.14	82.70	51.70
GPT-4o + VisEditAgent	**97.00**	**4.36**	**4.41**	**4.40**	**89.60**	**59.10**
Qwen3-VL-4B Zero-shot	80.54	3.11	3.41	3.44	68.13	38.91
Qwen3-VL-4B + VisEditAgent	**91.63**	**3.68**	**3.85**	**3.95**	**77.20**	**45.45**

Table 7: Human evaluation results for GPT-4o and Qwen3-VL-4B under zero-shot and VisEditAgent settings. GPT-4o is evaluated on all 1,395 examples, while Qwen3-VL-4B is evaluated on a 500-example stratified sample. Code execution and pass rate are reported as percentages. Task accuracy, readability and clarity, and visual quality use a 0–5 scale; visual similarity uses a 0–100 scale.

Metric	Pearson Correlation
Task Accuracy	83.38
Readability & Clarity	87.00
Visual Quality	86.50
Visual Similarity	86.00
Final Pass/Fail Agreement	**81.00%**

Table 8: Human–automatic evaluation agreement across all human-evaluated outputs. Pearson correlations are reported for scalar metrics; agreement is reported for the final pass/fail decision.

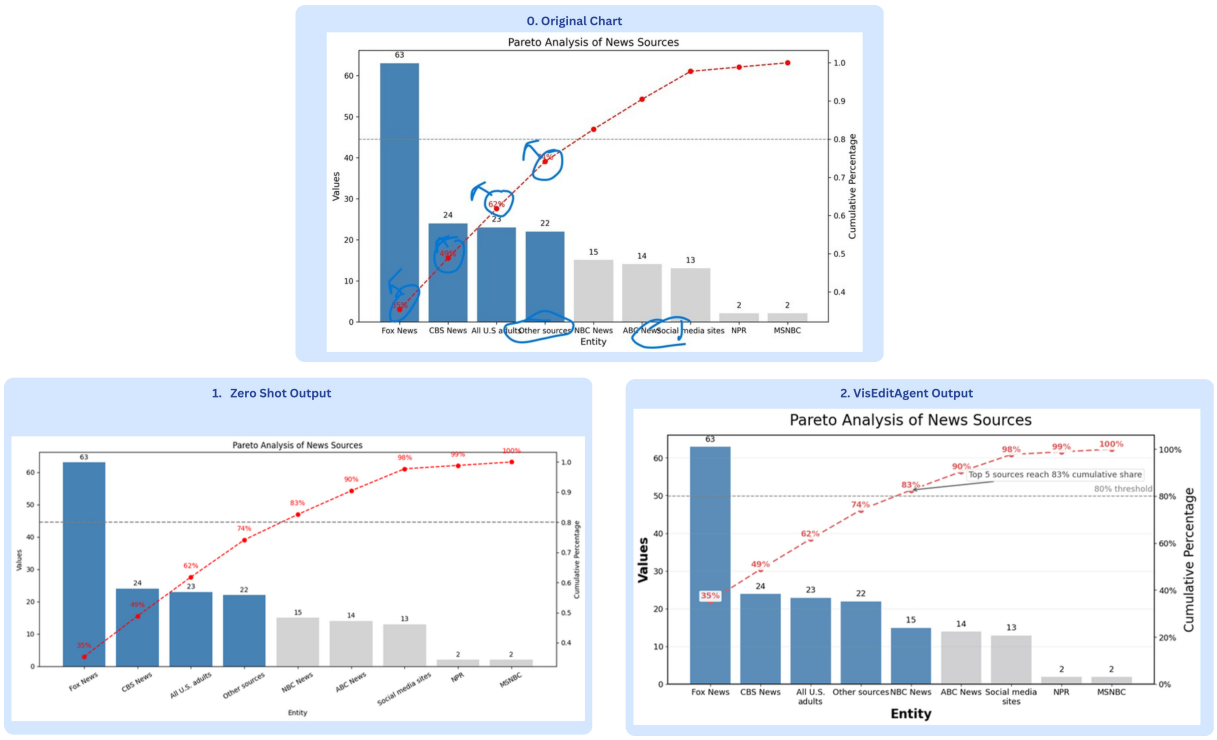


Figure 5: Qualitative example comparing GPT-4o zero-shot with GPT-4o + VisEditAgent. The original chart contains crowded x-axis labels and poorly positioned cumulative-percentage annotations. The editing instruction asks the model to improve label/annotation readability and spacing while preserving the chart semantics. Although GPT-4o zero-shot produces executable code, it only partially addresses these issues, whereas VisEditAgent produces a cleaner chart with better-positioned annotations and more readable labels.

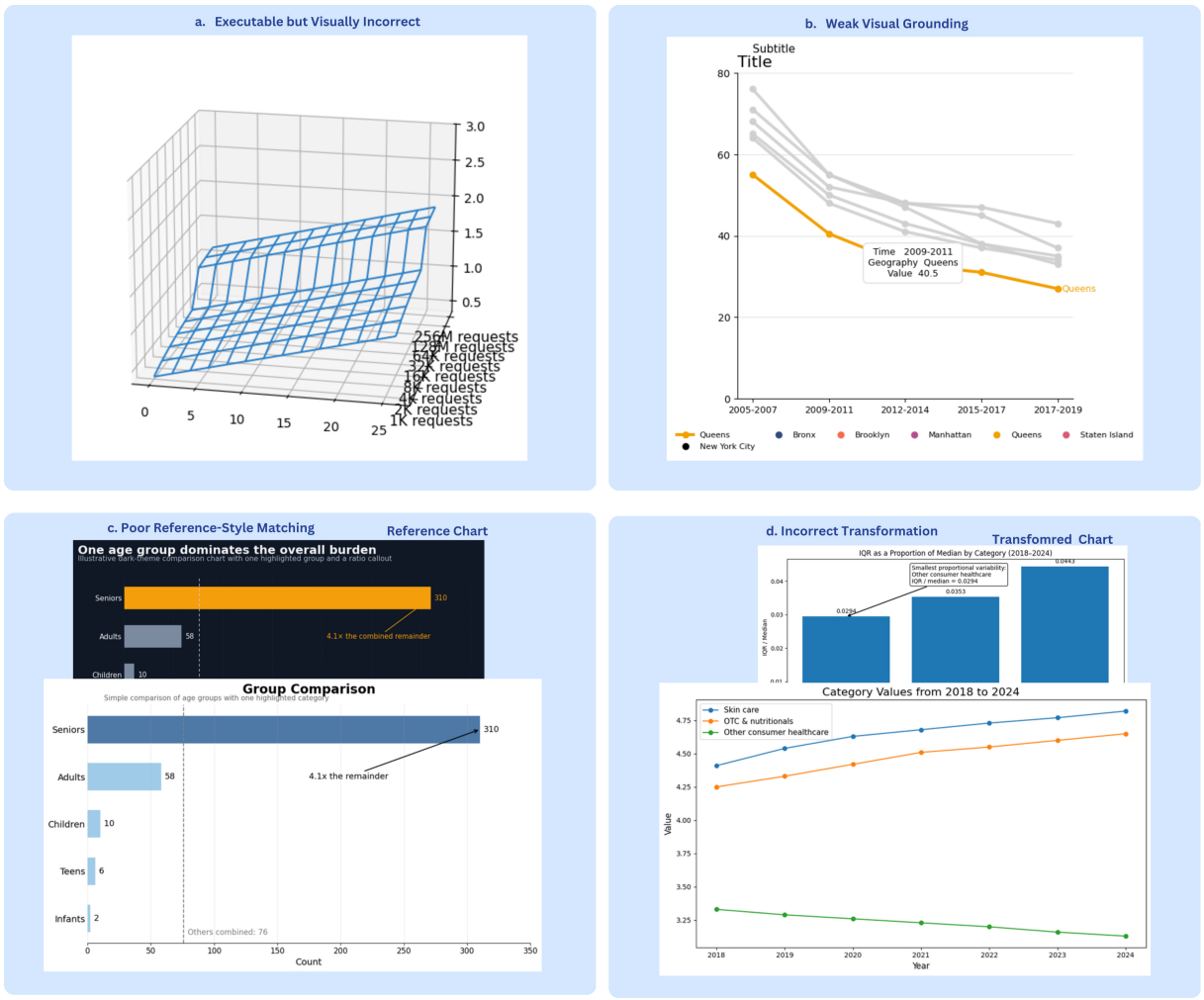


Figure 6: Error analysis examples showing that models often produce executable and visually plausible charts while missing the intended edit. (a) GPT-4o reduces clutter but the chart remains unreadable due to overlapping 1K–256M request labels. (b) GPT-4o fails to move the legend outside the line chart, leaving it over the plotted data. (c) InternVL-14B produces a clean chart but does not match the dark reference style. (d) Qwen3-VL-4B fails to transform the output into the reference visualization.

Metric	Rubric
Allowed Scores	For 0–5 metrics, the evaluator may use only the following scores: 0, 0.5, 1, 1.5, 2, 2.5, 3, 3.5, 4, 4.5, and 5. Visual similarity is scored from 0 to 100.
Task Accuracy	**Score range:** 0–5, in 0.5-point increments. **Correctness Repair and Quality Improvement:** 5 = fully fixes the issue shown in the image and instruction, preserves data meaning, and introduces no meaningful new problems; 4.5 = almost fully fixes the issue, with only tiny remaining issues; 4.0 = mostly fixes the issue, with minor remaining problems; 0 = invalid, irrelevant, unreadable, or does not address the requested fix at all. **All other intents:** 5 = closely matches the reference/target image and instruction, preserving data meaning unless the instruction asks otherwise; 4.5 = almost closely matches the reference/target, with only very minor differences; 4.0 = mostly matches the reference/target, with minor differences; 0 = invalid, irrelevant, unreadable, or does not match the reference/instruction at all.
Readability & Clarity	**Score range:** 0–5, in 0.5-point increments. 5 = labels, titles, axes, ticks, legends, fonts, and annotations are clear, readable, well-positioned, and non-overlapping. 0 = no valid readable chart, or chart elements are unreadable, clipped, or overlapped so badly that the chart cannot be interpreted.
Visual Quality	**Score range:** 0–5, in 0.5-point increments. 5 = polished, professional, visually clean, accessible, uses appropriate colors/scales/chart type, and communicates the intended information clearly. 0 = no valid visualization, irrelevant image, misleading visual design, or extremely poor visual quality.
Visual Similarity	**Score range:** 0–100. **Correctness Repair and Quality Improvement:** Compare the rendered chart to the clean corrected version of the original buggy/marked chart. The rendered chart should remain visually similar to the original chart, but should fix the issue described in the instruction. Do not penalize the rendered chart for not reproducing handwritten marks or the original bug. **Style Adaptation:** Compare style similarity: colors, fonts, background, gridlines, mark style, layout, and overall appearance. Data values may differ if the task uses the base data, but the style should match the reference image closely. **Style-Aware Error Repair:** The reference image is the target chart style/output and may show marked potential errors. The rendered chart should recreate the clean target chart while avoiding the marked or anticipated issues. Do not reward reproducing handwritten marks or buggy artifacts. **All other intents:** The reference image is the target chart. The rendered chart should match it as closely as possible in chart type, data, encodings, colors, labels, title, axes, legend, annotations, layout, and overall appearance.
Visual Similarity Scale	100 = near-perfect match under the intent-specific rules; 90–99 = almost identical, only tiny differences; 80–89 = good match but minor differences; 60–79 = partially correct but noticeable differences; 40–59 = weak match; 1–39 = barely related; 0 = completely different, invalid, or no usable chart.
Intermediate Guide for 0–5 Metrics	4.5 = very good; only tiny issues. 4.0 = good; small issues but clearly successful. 3.5 = decent; mostly usable but with noticeable issues. 3.0 = average; important issues affect clarity/correctness. 2.5 = below average; several issues may mislead or confuse. 2.0 = poor; significant issues. 1.5 = very poor; major problems. 1.0 = barely usable or mostly wrong. 0.5 = almost completely failed but has a tiny relevant element. 0.0 = failed.
Evaluator Rules	If executable = 0, all scores must be 0. The evaluator is instructed to be strict. Handwritten annotations, circles, arrows, scribbles, highlights, and notes are ignored unless they describe what should be fixed. A nice-looking chart is not rewarded if it fails the instruction. A chart that changes data meaning is not rewarded unless requested.

Table 9: Evaluation rubric used for scoring rendered visualization outputs. Task accuracy, readability and clarity, and visual quality are scored on a 0–5 scale in 0.5-point increments. Visual similarity is scored on a 0–100 scale. The rubric is intent-aware and distinguishes repair, style adaptation, style-aware error repair, and other reference-guided target-matching tasks.

Component	Configuration
Inference parameters	For direct model inference, each model receives the visualization image and the corresponding editing prompt as multimodal input. The model is instructed to return only complete revised code, without markdown or explanation. Unless otherwise stated, we use image detail = `high`, temperature = 0.2, top-p = 1.0, maximum output tokens = 4096, and output format = complete executable visualization code only.
Evaluation parameters	Generated code is cleaned, executed, and rendered before scoring. Vega-Lite specifications are rendered using `vl-convert-python`, while Python/Matplotlib outputs are executed using a non-interactive `Agg` backend. Non-executable outputs are assigned executability 0 and zero scores for all remaining metrics. GPT-4o is used as the default evaluator; for GPT-4o outputs, Gemini 2.5 Pro is used to avoid self-evaluation bias. The evaluator uses temperature = 0.0, maximum output tokens = 1200, and image detail = `high`. Task accuracy, readability and clarity, and visual quality are scored on a 0–5 scale in 0.5-point increments; visual similarity is scored on a 0–100 scale.
Final pass criteria	An output is counted as a pass only if all conditions are satisfied: `executable = 1`, task accuracy ≥ 4.5, readability and clarity ≥ 4.0, visual quality ≥ 4.0, and visual similarity ≥ 90.

Table 10: Inference and evaluation parameters used in VisEditBench experiments. These settings are fixed across model evaluations unless otherwise stated.

Difficulty Classification Prompt

You are an expert annotator for VisEditBench, a benchmark for visualization code editing from multimodal feedback.
Your task is to classify one visualization editing sample as **Easy**, **Medium**, or **Hard**.
You will be given:

1. The input visualization code.
2. The input chart image, which may show a buggy chart, a human-marked chart, or a reference/target chart.
3. The natural-language editing instruction.
4. Metadata such as editing intent, problem type, chart type, and visualization library, if available.

Classify the task difficulty based on the expected effort required to produce the correct edited visualization code while preserving the intended data semantics. Use the number of distinct issues as the main signal, while also considering visual reasoning, code reasoning, issue interaction, scope of required edits, reference-style matching complexity, and risk of changing the underlying data meaning.

Easy. A task is Easy if it contains one main issue and the required edit is local, direct, and clearly specified. The issue is usually visible from the chart and can be fixed by modifying one chart property or a small number of nearby parameters. The correct code location is usually obvious, and there is little risk of changing the underlying data semantics.
Typical Easy tasks include:

- increasing font size or rotating tick labels;
- moving a legend, title, or axis label;
- fixing one simple label overlap;
- changing a single color, marker, line style, or font property;
- adjusting figure size, margins, or axis-label spacing;
- adding or removing one simple annotation.

Choose Easy when the task mostly requires fixing one localized issue with minimal visual-code reasoning.

Medium. A task is Medium if it contains two distinct issues, or if one issue requires coordinated changes across multiple chart components. The task may require moderate reasoning over the image, instruction, and code, but the required edits are still reasonably well-scoped. Medium tasks do not need to be structurally complex; they only need to require more than one localized change or moderate visual-code alignment.
Typical Medium tasks include:

- fixing two visible issues in the same chart;
- fixing multiple overlapping labels, ticks, or annotations;
- adjusting axes, legends, titles, layout, and spacing together;
- changing encodings while preserving the chart's meaning;
- improving readability across several chart components;
- matching a reference style with several visible properties such as colors, fonts, gridlines, and layout;
- modifying grouped, stacked, or multi-series charts.

Choose Medium when the task requires two fixes, several coordinated edits, or moderate visual-code reasoning, but does not require major structural transformation or highly complex semantic reasoning.

Hard. A task is Hard if it contains three or more distinct issues, multiple interacting causes, structural code changes, complex reference matching, or careful preservation of data semantics while making nontrivial visual modifications. A task can be Hard even if the instruction is short, when the correct solution requires several dependent edits or substantial reasoning about both the rendered image and the code.
Typical Hard tasks include:

- fixing three or more visible issues in the same chart;
- converting between chart types while preserving analytical meaning;
- repairing a chart with multiple interacting errors;
- matching a complex reference style involving layout, colors, fonts, annotations, legends, background, and mark design;
- modifying faceted, layered, composite, or multi-panel visualizations;
- fixing issues caused by both data transformation and visual encoding;
- coordinating changes across scales, legends, annotations, encodings, layout, and data processing.

Choose Hard when the task requires three or more fixes, interacting fixes, structural transformation, complex reference matching, or high risk of changing data semantics.

Issue-count guideline. Use the number of distinct issues as the main difficulty signal, while still considering reasoning complexity. One localized issue usually suggests Easy, two distinct issues usually suggest Medium, and three or more distinct issues usually suggest Hard. However, override this rule when complexity clearly changes the difficulty: a single issue can be Medium or Hard if it requires structural code changes, complex reference matching, nontrivial data transformation, or high risk of changing data semantics; multiple issues can be lower difficulty if they are trivial, independent, and solved by the same localized edit.

Decision rules. Focus on the difficulty of producing the correct edited code, not merely understanding the instruction. Do not label a task Hard only because the chart looks visually busy, and do not label a task Easy only because the instruction is short.....

Return only valid JSON:

```
{
  "difficulty":  "Easy" | "Medium" | "Hard",
  "explanation":  "One or two sentences explaining the label."
}
```

Figure 7: Prompt used for difficulty classification. GPT-5 was used to classify each VisEditBench example as Easy, Medium, or Hard under this rubric.

Method	Cand.	Render	Refine	Pass Rate
GPT-4o Zero-shot	1	No	No	55.0
Best-of-3	3	No	No	58.2
Best-of-5	5	No	No	59.4
3-candidate Code-only Refinement	3	No	Yes	62.8
GPT-4o + VisEditAgent	**3**	**Yes**	**Yes**	**68.0**

Table 11: Compute-matched comparison of VisEditAgent on a stratified 500-task subset of VisEditBench. Pass rates are percentages (%).

Model	Easy	Medium	Hard	Overall
GPT-4o	69.9	54.8	37.8	55.8
GPT-5-Mini	73.1	60.2	42.0	60.0
GPT-5	78.1	67.1	47.0	65.7
Gemini-3-Flash	26.0	14.9	7.9	17.1
Claude-4.5-Sonnet	75.2	64.1	44.1	62.7
Claude-4.6-Sonnet	**86.2**	**76.3**	**56.2**	**74.5**

Table 12: Final pass rate (%) by task difficulty.

Model	Bar	Line	Scatter	Boxplot	Overall
GPT-4o	57.5	56.3	48.1	61.1	55.8
GPT-5-Mini	60.1	59.9	65.4	51.9	60.0
GPT-5	65.2	67.1	57.7	72.2	65.7
Gemini-3-Flash	18.6	15.3	23.1	16.7	17.1
Claude-4.5-Sonnet	62.0	63.4	69.2	57.4	62.7
Claude-4.6-Sonnet	**73.1**	**76.8**	**80.8**	**68.5**	**74.5**

Table 13: Final pass rate (%) by chart type.